Davante

7

PROGRAMACIÓN

Introducción a la orientación a objetos

ÍNDICE

/ 1. Introducción y contextualización práctica 3

/ 2. Programación orientada a objetos 4

2.1. Principios básicos de la POO 5

2.2. Ventajas e inconvenientes de la POO 5

2.3. Protección de datos en la POO 6

/ 3. Caso práctico 1: "¿Programación orientada a objetos o uso de variables?" 6

/ 4. Clases 7

4.1. Características y componentes de una clase 8

4.2. Atributos de una clase 9

/ 5. Relación entre clases 10

5.1. Relación de asociación 11

5.2. Relación de dependencia 12

5.3. Relación de generalización 12

/ 6. Caso práctico 2: "Relaciones entre clases" 13

/ 7. Creación de clases 14

/ 8. Documentación de clases 15

/ 9. Resumen y resolución del caso práctico de la unidad 16

/ 10. Bibliografía 17

© MEDAC ISBN: 978-84-19885-69-2
Reservados todos los derechos. Queda rigurosamente prohibida, sin la autorización escrita de los titulares del copyright, bajo las sanciones establecidas en las leyes, la reproducción, transmisión y distribución total o parcial de esta obra por cualquier medio o procedimiento, incluidos la reprografía y el tratamiento informático.

OBJETIVOS

Conocer los principios básicos de la programación orientada a objetos.

Conocer el concepto de encapsulado de información.

Distinguir los conceptos de clase y objeto.

Diferenciar entre variable y objeto.

Distinguir los tipos de relaciones entre clases.

Documentar una clase con Doxygen.

/ 1. Introducción y contextualización práctica

Este tema se centra en la introducción a los conceptos básicos de las clases y los objetos, y en sus grandes ventajas a la hora de crear código.

Estos conceptos básicos son tanto a nivel de protección de información como de encapsulamiento y abstracción de los datos, donde la idea de clases y objetos van a ser fundamentales, ya que se verá cómo implementar la programación dirigida a objetos mediante la creación de clases.

También es importante explicar las distintas relaciones entre clases, puesto que no son independientes, sino que viven interconectadas.

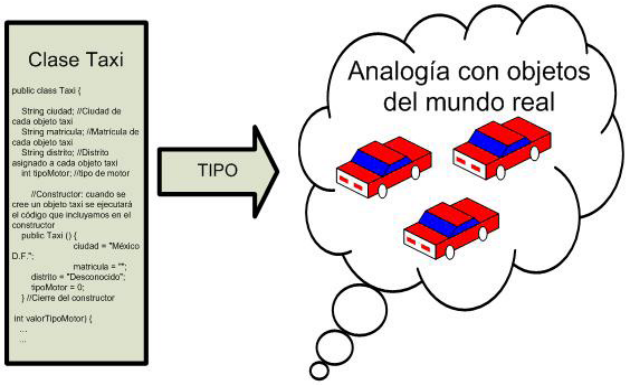

Fig.1. Representación del taxi del mundo real.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema. Encontrarás su resolución en el apartado 'Resumen y resolución del caso práctico'.

Audio intro. "Creación de libros"

https://on.soundcloud.com/euRMKCVsuSFjT3jZ9

/ 2. Programación orientada a objetos

Cuando surgieron los primeros lenguajes de programación, se utilizaba el paradigma de la programación estructurada, el cual permitía realizar cualquier programa mediante las estructuras básicas, que son:

- Las secuencias (instrucciones simples).
- Las instrucciones condicionales.
- Las instrucciones repetitivas o bucles.

Con el paso del tiempo, este paradigma se quedó pequeño, los programas cada vez eran mayores en cuanto a líneas de código y complejidad, con lo que su creación, entendimiento y mantenimiento se hacían cada vez más difíciles. Todo esto terminó haciendo que se originara un nuevo paradigma en el mundo de la programación, la programación dirigida a objetos.

La **programación dirigida a objetos (POO)** es un nuevo enfoque que **intenta representar un sistema real mediante un modelo software, abstrayendo las partes más importantes del sistema y materializándolas en objetos que se puedan manipular**. Lo complicado es encontrar dichos objetos, por lo que la parte de diseño más importante en el desarrollo del software orientado a objetos es la definición de dichos objetos.

En la programación estructurada, los datos y las operaciones estaban separados. Por un lado, teníamos las variables (datos) y, por otro, las instrucciones (funciones). Este enfoque busca las funciones necesarias para resolver el problema y los datos sobre los que operan las funciones, como si se tratase del desarrollo de una determinada tarea.

La POO busca los objetos y establece conexiones entre ellos, es decir, **qué operaciones necesitan realizarse y qué es lo que resulta de la aplicación de dichas operaciones**. Así, el sistema se compone de objetos, formados por sus datos y las operaciones que pueden realizar. La ejecución del sistema es, por tanto, un proceso de comunicación mediante objetos.

Los objetos se relacionan con otros para elaborar modelos. Las relaciones permiten que los objetos puedan relacionarse con otros que forman parte del sistema. Las relaciones más importantes en la POO son las llamadas **relaciones jerárquicas**.

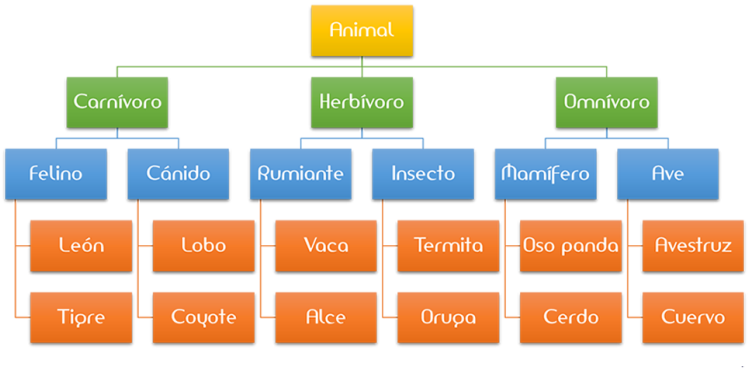

Fig.2. Relación jerárquica.

¡! Sabías que...

Las relaciones se basan en el concepto de herencia, es decir, en las relaciones padre-hijo entre objetos. Los objetos hijos heredan de sus padres algunas de sus características.

2.1. Principios básicos de la POO

La POO nos va a proporcionar los siguientes principios:

- **Abstracción**: Consigue que cada objeto utilizado funcione como un único agente, teniendo sus propias características y funciones, independientemente de cómo esté construido.
- **Encapsulamiento**: Proporciona cohesión en los datos que forman un objeto, ya que esos datos estarán encapsulados dentro del objeto, aunque no los veamos.
- **Herencia**: El concepto de herencia se basa en las relaciones existentes entre las clases, pudiendo estas formar una relación jerárquica, donde haya clases que heredan comportamiento de otras.
- **Polimorfismo**: Proporciona a un objeto la posibilidad de tener diferentes comportamientos que están asociados a diferentes objetos. Este concepto va ligado al de herencia.

2.2. Ventajas e inconvenientes de la POO

La razón de que la POO goza de tanta popularidad actualmente es, precisamente por esto, porque los programas se encuentran conformados por **unidades aisladas y bien definidas**, lo que deriva en los siguientes **beneficios**:

- Favorece la reutilización.
- Aumento del nivel de abstracción.
- Acelerar la fase de desarrollo.
- Facilitar el mantenimiento.
- Permitir la creación de sistemas complejos.
- Proteger la información.

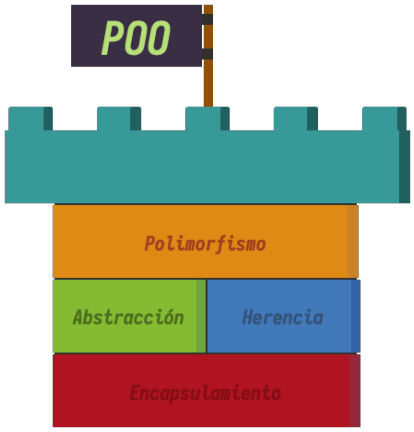

Fig.3. Pilares de la POO.

Por otro lado, tiene las siguientes **desventajas**:

- El tiempo de ejecución del mecanismo de vinculación dinámica (polimorfismo) es lento.
- Mayor complejidad, ya que la implementación requiere realizar un análisis más exhaustivo de la aplicación.
- Dificultad de aprendizaje, puesto que es necesario aprender las librerías de clase que los lenguajes incorporan, para utilizar la clase que deseamos.

2.3. Protección de datos en la POO

Un concepto muy importante en la programación orientada a objetos es la protección de datos. Este punto hace alusión directa a los conceptos de **encapsulación** y **ocultación** de la información.

Si recordamos, el hecho de **encapsular** información quiere decir que los datos que componen un objeto no serán visibles desde fuera, teniendo que usar métodos concretos para poder visualizarlos y modificarlos. Por tanto, este proceso solo se encarga de mostrar las operaciones disponibles o la interfaz pública para interactuar con él. Esto nos proporciona un nivel de protección de datos bastante alto, ya que desde fuera no se podrá acceder a la forma en la que se ha diseñado el objeto, evitando así posibles problemas futuros.

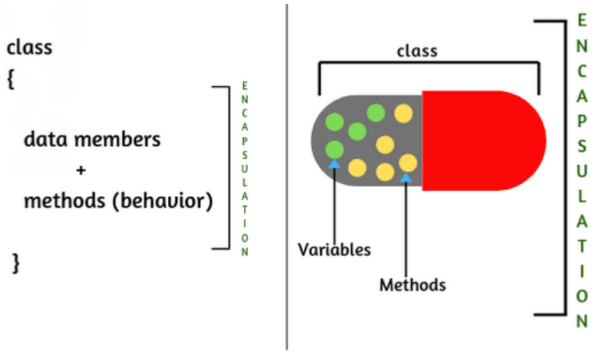

Fig.4. Encapsulación.

El principio de **ocultación** de información nos dice que podemos aplicar a las variables de nuestros objetos diferentes modificadores de acceso en cuanto a la visibilidad en el exterior se refiere.

Básicamente, podremos hacer que sean visibles o no, teniendo así el poder de decidir qué partes de nuestros objetos dejamos ver desde el exterior.

Todo esto nos va a facilitar trabajar con objetos complejos, ya que no necesitaremos conocer su implementación interna, tan solo qué funcionalidades tiene, qué parámetros de entrada requiere y qué nos devolverá.

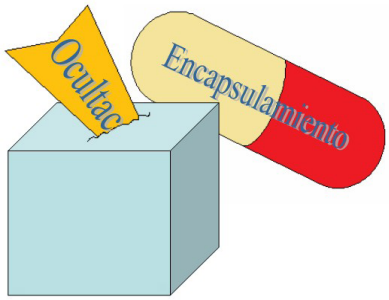

Fig.5. Ocultación.

La encapsulación y ocultación de la información ayudan a crear clases más robustas y mantenibles al reducir la dependencia de otras clases en los detalles de implementación interna de una clase y al proporcionar un mecanismo controlado para acceder y modificar los datos de un objeto.

/ 3. Caso práctico 1: "¿Programación orientada a objetos o uso de variables?"

Planteamiento: La programación orientada a objetos (POO) es crucial porque mejora la organización, mantenibilidad, escalabilidad del código, facilita la colaboración y el desarrollo en equipo, y proporciona una manera natural de modelar problemas complejos.

Nudo: Se está realizando un programa en el que se tienen que representar los alumnos de un instituto para poder hacer gestiones con algunos de sus datos: darlos de alta, baja, hacer consultas, etc.

Cada alumno tiene un conjunto de datos, como son nombre, apellidos, DNI, edad, curso, asignaturas, etc. En ese momento, entran en duda, ¿sería mejor crear variables para representar esos datos, o se debería utilizar la POO?

¿Qué piensas sobre ello? ¿Qué forma crees que será la mejor para resolver este problema?

Desenlace: Cuando es necesario representar un dato del mundo real, en este caso un alumno, lo más fácil es utilizar la POO, ya que gracias a sus principios, como la encapsulación y protección de datos, el problema se simplifica muchísimo en comparación al uso de variables independiente para representar cada una de las propiedades de los alumnos. Piensa que si a un alumno lo representan 10 datos diferentes, ¿cuántos datos necesitaremos para un instituto de 1.000 alumnos, que es lo normal que se puede dar?

Mediante la programación orientada a objetos, con la creación de una única clase que modele las distintas propiedades del alumno, se resolvería ese trabajo.

Fig.6. Piensa en cómo diseñar los datos.

/ 4. Clases

El uso de clases está inspirado en la abstracción y representación del mundo real en un programa informático, para lo que es necesario el uso y la clasificación de objetos en grupos.

Un ejemplo podría ser que todos los animales que comen carnes se engloban dentro del grupo de los carnívoros, y, de manera análoga, con los animales herbívoros y omnívoros que, a su vez, pertenecen al grupo de animales.

Otro ejemplo podría ser José, Óscar y Francisco. Todos ellos son nombres que pertenecen al mismo grupo: son personas. Cada uno de estos nombres representa un objeto que pertenece al mismo grupo de personas y es a este grupo a lo que denominamos clase.

En este caso, cada persona tiene solo un nombre, aunque pueda tener más propiedades, como son el apellido o la edad, pero al realizar la abstracción, nosotros definimos las características que nos interesan para resolver el problema.

Para detectar cuándo necesitamos crear una clase, nos tenemos que preguntar lo siguiente: ¿Esto existe en el mundo real? ¿Tiene propiedades? ¿Puede realizar acciones?

Un ejemplo muy claro de esto es una persona, y en respuesta a las preguntas, tenemos:

- ¿Esto existe en el mundo real? Sí, las personas existen el mundo real.
- ¿Tiene **propiedades**? Sí, una persona tiene un DNI, un nombre, unos apellidos, una edad...
- ¿Puede realizar **acciones**? Sí, una persona puede saludar, andar...

Si cumple con las tres preguntas, es un caso claro de clase; si, por el contrario, no cumple con alguna de ellas, probablemente no sea un candidato a poder ser representado mediante una clase.

En otras palabras, una clase es un modelo que describe los **atributos** y el **comportamiento** que tendrán los objetos que se creen a partir de ella.

Recuerda...

Los comportamientos de una clase definen cómo interactúa con el mundo exterior y son fundamentales para el diseño y desarrollo de programas.

La figura 7 muestra las propiedades y los comportamientos de un perro.

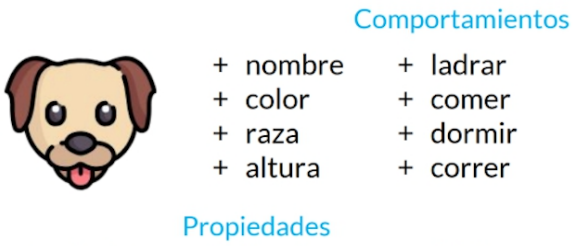

Fig.7. Propiedades y comportamiento de la clase Perro.

4.1. Características y componentes de una clase

Una clase es un concepto abstracto que sirve para definir una generalización de un tipo determinado de objetos mediante una plantilla. A la hora de nombrarla es igual que las variables, pero, como norma no obligatoria, deberían empezar por una letra **mayúscula**.

Se puede decir, por tanto, que:

- Una clase es un modelo a partir del cual se pueden construir tantos elementos como se quiera.
- Las clases no son utilizables directamente. Su función es crear instancias u objetos de la clase.
- Dos instancias diferentes de una misma clase comparten el mismo conjunto de métodos y la misma lista de datos, pero con valores diferentes.

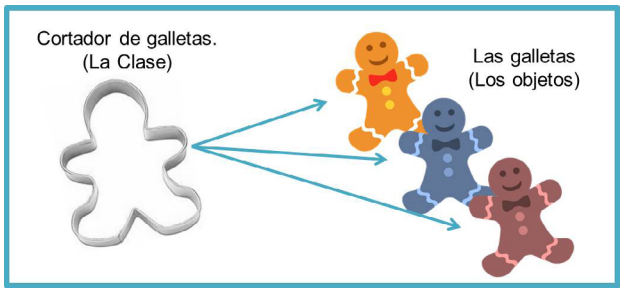

Fig.8. Clases y objetos.

Respecto a los componentes que conforman una clase son:

- **Datos de una clase**: Determina los tipos de datos que se van a guardar de cada objeto, así, los objetos se diferenciarán unos de otros en función del valor que tenga para dichos datos. Se le suele denominar **atributos**. En el ejemplo anterior, nombre, color, raza y altura serían los datos de la clase Perro.
- **Métodos de una clase**: Son los métodos que pueden ser llamados por los objetos para realizar operaciones con sus datos. Los tipos de métodos que podemos encontrar son:
  - » **Observadores**: Sirven para obtener información del objeto. En el ejemplo anterior, ladrar, comer, dormir y correr.
  - » **Modificadores**: Son utilizados para modificar datos de los objetos.

» **Constructores**: Son un tipo específico de método empleado para asignar valores a los datos de un objeto en el momento de su creación.

» **Destructores**: Son ejecutados cuando un objeto va a ser eliminado.

¡! Sabías que...

En Java, no existe un destructor como tal, como se encuentra en otros lenguajes de programación como C++. En su lugar, Java cuenta con un mecanismo de recolección de basura (garbage collection), que automáticamente libera la memoria de los objetos cuando ya no son referenciados.

4.2. Atributos de una clase

Los datos o propiedades que definen la clase se denominan atributos. Si partimos de un proyecto llamado animales en el que tenemos que crear una nueva clase Perro, y nos piden definir sus atributos: nombre, color, raza y altura, podemos observar la manera de definir los atributos de una clase en la figura 9:

```
class Perro {
        String nombre;
        String color;
        String raza;
        double altura;
}
```

Fig.9. Declaración de atributos.

Donde el tipo que determina cada atributo puede ser de cualquier tipo primitivo o una clase, como se verá más adelante. El código empleado para definir nuestra clase Perro será el siguiente:

Como se puede observar, es posible declarar distintos tipos dentro de una clase.

Una vez definidos los atributos, se le puede asignar un valor por defecto a cada uno de ellos en la misma declaración, como se puede ver en la siguiente figura 10:

```
Class NombreClase {
        tipo atributo1 = valor;
        tipo atributo2 = valor;
        ......
}
```

Fig.10. Declaración y asignación de valores de atributos.

Estos atributos pueden cambiar de valor a lo largo de la ejecución del programa, salvo que sean definidos como constantes mediante el uso del modificador **final**, en cuyo caso, una vez inicializado, no podrá cambiar de valor durante su ejecución.

```
package animales;

/**
 *
 * @author franciscojimenezmoral
 */
public class Perro {

    final String nombre = "Nerón";
    String color;
    String raza;
    double altura;

}
```

Fig.11. Declaración de atributos de la clase Perro.

Audio 1. "Diferencia entre variables y objetos"

https://bit.ly/38Z9swQ

/ 5. Relación entre clases

A la hora de afrontar un problema orientado a objetos, el primer paso es identificar las clases necesarias que formar parte de la solución, y una vez identificadas, se debe pensar la interconexión que existe entre dichas clases. Las clases no viven aisladas, sino que viven interconectadas con otras clases, esa interconexión es lo que se denomina **relación**.

Por tanto, la solución al problema pasa por identificar las clases que lo forman y su relación, siendo una labor crucial su identificación para una solución eficiente.

Estas relaciones se pueden deber a varios factores, como puede ser un comportamiento común o la necesidad que tenga una clase de usar otra clase. Eso no quita que existan clases aisladas que no interactúen con otras, aunque la mayoría de ellas no puedan y deban cooperar con otras.

Entre las razones que promueven las relaciones entre clases en la POO se encuentra:

- Modelado de la realidad.
- Reutilización de código.
- Abstracción y encapsulación.
- Flexibilidad y extensibilidad.
- Mantenimiento y evolución del código.
- Separación de responsabilidades.

Se trata de un apartado fundamental para el diseño y la implementación de sistemas software robustos y mantenibles en el tiempo al proporcionar una manera estructurada y flexible de organizar los distintos componentes de un sistema.

Un ejemplo podría ser una casa formada por otras entidades, como la pared, el suelo o el techo, que, a su vez, contiene elementos de uso, como son puertas o ventanas.

Estas relaciones entre clases podemos clasificarlas en tres tipos según muestra la figura 12:

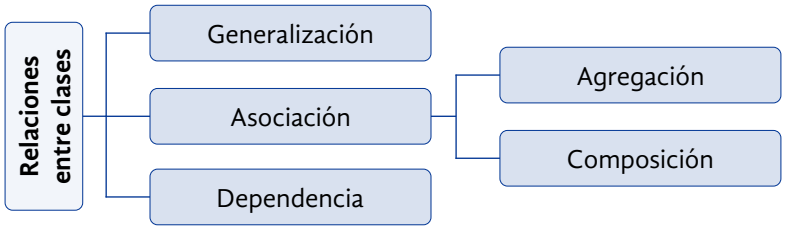

Fig.12. Tipos de relaciones entre clases.

5.1. Relación de asociación

La relación de asociación en POO se refiere a la estructura en la que una clase está compuesta por objetos de otras clases, siendo su predicado '**tiene-un**' o '**parte-de**'. Por ejemplo, en la figura 13, una clase Coche podría tener objetos de las clases Puerta, Rueda, etc., como atributos.

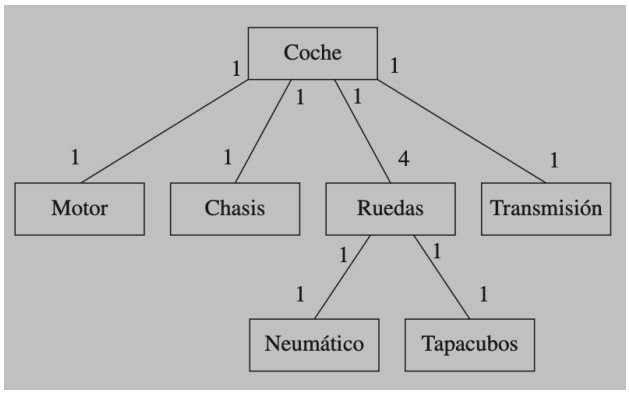

Fig.13. Relación de asociación de la clase Coche.

Esta relación puede ser binaria si conecta dos elementos o n-aria si involucra más de dos elementos.

Este tipo de relaciones pueden ser bidireccionales si se recorren en ambos sentidos o unidireccionales en caso de restringir la dirección en un único sentido.

En UML, una asociación se representa con una línea continua que conecta las clases relacionadas. En el caso de ser unidireccional, la línea continua termina con una punta de flecha, indicando el sentido de la asociación. Además, se puede indicar la multiplicidad para denotar cuántos objetos de una clase están conectados en esta relación. La tabla 1 representa los distintos tipos de multiplicidad:

Multiplicidad	Significado
1	Uno y solo uno
0 .. 1	Cero o uno
N .. M	Desde N hasta M
*****	Cero o varios
0 .. *	Cero o varios
1 .. *	Uno o varios

Tabla 1. Multiplicidad de las asociaciones.

Dentro de la relación de asociación se pueden distinguir los siguientes casos:

- **Relaciones involutivas**: Cuando la misma clase aparece en los dos extremos de la asociación, por ejemplo, al casarse una persona con otra persona o un empleado dirige a otro empleado.
- **Relación de agregación**: Las partes pueden formar parte de los distintos agregados, como es el caso de un punto en un polígono.
- **Relación de composición**: Las partes solo existen asociadas al compuesto.

5.2. Relación de dependencia

Se trata de una relación temporal y funcional donde un objeto usa a otro, y una vez concluya esa utilización, se acaba la relación entre ambas clases, que se reconoce por el predicado '**usa_a**' o '**trabaja_con**'.

En esta relación existe una clase dependiente y otra independiente, donde obviamente la clase dependiente necesita a la independiente para realizar una función.

En UML, esta relación se representa con una flecha discontinua dirigida hacia el elemento del que depende, con la punta de la flecha apuntando hacia la clase independiente. Las dependencias se usan para indicar que un elemento utiliza a otro. Por ejemplo, en la figura 14, el profesor depende de la tiza para dar clase.

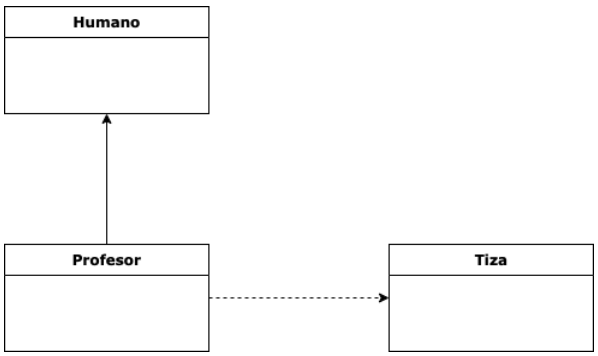

Fig.14. Relación de dependencia.

5.3. Relación de generalización

Hay que distinguir entre dos clases con distintos niveles de abstracción. La clase de nivel más alto se denomina clase Padre o superclase, y la de menor nivel de abstracción es la clase Hija. Esto se evidencia cuando una clase es un subtipo de otra (también llamada *a-kind-of*) y se identifica con el predicado '**es-un**'.

La clase Hija hereda los atributos y métodos de la clase Padre, pudiendo agregar los suyos propios. Es crucial diferenciar entre objetos de una clase y clases que heredan de otras.

Por ejemplo, un oso es un mamífero, donde 'mamífero' es la superclase y 'oso' es la subclase. Otros ejemplos son una rosa, que es una flor, o un empleado, que es una persona. Esta relación se conoce como herencia y es una característica clave de la POO.

En UML, la herencia se representa con flechas apuntando a la clase Padre, generando jerarquías entre clases, como se muestra en la figura 15.

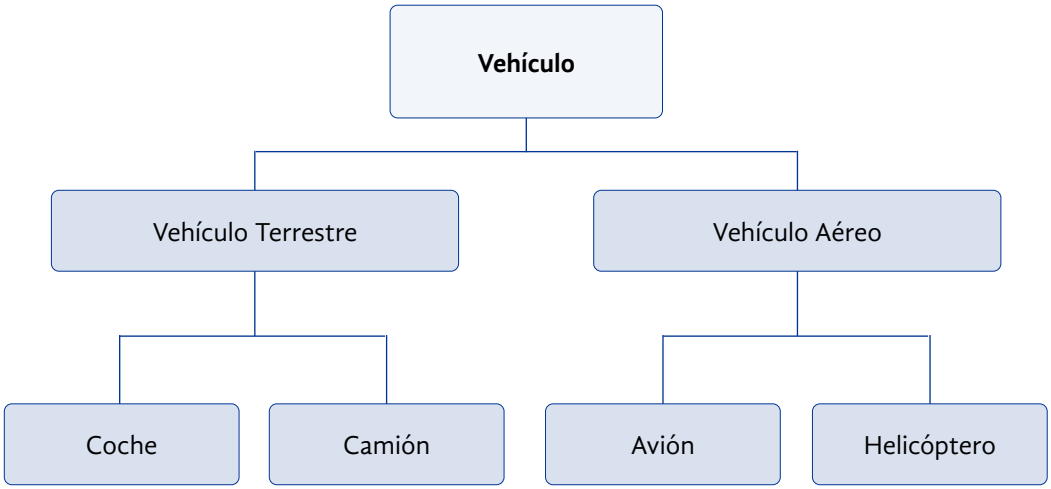

Fig.15. Relación de generalización.

/ 6. Caso práctico 2: "Relaciones entre clases"

Planteamiento: Las relaciones entre clases son fundamentales en la programación orientada a objetos (POO) porque permiten modelar de manera efectiva y eficiente sistemas complejos, proporcionando claridad, modularidad y reutilización del código.

Nudo: Dados los siguientes pares de clases:

Pares de clases	
Coche	**Motor**
Conductor	**Vehículo**
Perro	**Animal**

Tabla 2. Pares de clases.

Describe qué tipo de relación existe entre ellas y justifica la respuesta.

Desenlace:

- Clase Coche – Clase Motor: **Dependencia**: Coche tiene una dependencia con Motor porque **necesita** usar Motor para funcionar.
- Clase Conductor – Clase Vehículo: **Asociación**: Vehículo tiene una asociación con Conductor. Vehículo **tiene un** Conductor, mostrando una relación unidireccional.
- Clase Perro – Clase Animal: **Generalización-herencia**: Perro hereda de Animal. El perro **es un** animal.

/ 7. Creación de clases

A la hora de crear una clase en Netbeans, crearemos un fichero independiente que deberá llamarse igual al nombre de la clase y tendrá extensión **.java**.

Los pasos son los siguientes:

- Pulsar sobre el menú File -> New File y se mostrará una ventana como la que se ve a continuación:

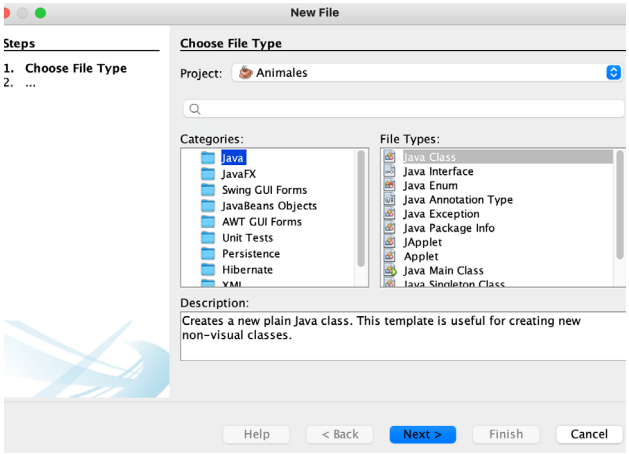

Fig.16. Elección del tipo de fichero.

- Sobre esta ventana se selecciona el proyecto en el que se quiere añadir la clase, en este caso, ha sido sobre el proyecto Animales, y el siguiente paso es seleccionar el tipo de fichero y vendrá seleccionado por defecto Java Class.

- Pulsamos el botón Next y accedemos a la siguiente ventana que se muestra en la figura 17.

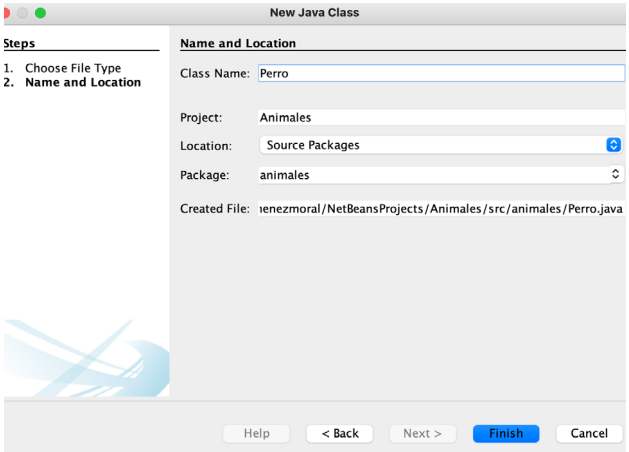

Fig.17. Nombre y ubicación del fichero.

- En esta ventana, podremos definir el nombre de la clase y la ubicación, y al pulsar sobre finalizar, se cerrará la ventana y nos mostrará el fichero creado.

```
package animales;

/**
 *
 * @author franciscojimenezmoral
 */
public class Perro {

}
```

Fig.18. Fichero Perro.

Vídeo 1. "Creación de una clase y un objeto"

https://bit.ly/2C4lRTW

/ 8. Documentación de clases

La documentación del código es una parte fundamental en cualquier programa, independientemente del lenguaje de programación que se utilice.

Cuando documentamos código, estamos explicando cuál es la funcionalidad del mismo, para así evitar tener que leer y entender el código que hay escrito.

Entre las **razones por las que se considera esencial la documentación**, se encuentran:

- Claridad y comprensión.
- Facilita la colaboración.
- Acelera el desarrollo.
- Mejora la mantenibilidad.
- Facilita su depuración.
- Promueve la reutilización de código.
- Ayuda a usuarios finales.

Una de las formas más fáciles de documentar un programa es mediante **Doxygen**. Doxygen significa generador de documentación para código fuente, y viene de **dox** (documento) y **gen** (generador).

Doxygen es un generador de código automático a partir del código fuente de un proyecto que está disponible para infinidad de lenguajes de programación, Java entre ellos.

Una de las cosas que podremos documentar con Doxygen serán las clases. Para ello, vamos a utilizar las siguientes etiquetas:

- **@version**: Esta etiqueta indica la versión de la clase.
- **@author**: Esta etiqueta indica quién es el autor de la clase.

Todo el código de documentación Doxygen deberá ir entre **/**** y ***/**.

Siguiendo el ejemplo de la clase Coche, podríamos documentarla de la siguiente forma:

```
**
 * Esta clase representa un coche
 * @version 1.0
 * @author Francisco Jiménez Moral
 ** /

public class Coche {
    //Atributos de la clase
    private String matricula, marca, modelo;
    private double precio;
}
```

Fig.19. Documentación con Doxygen.

Entre las funcionalidades que incluye esta herramienta, se encuentra la de realizar un diagrama de clases UML.

/ 9. Resumen y resolución del caso práctico de la unidad

La programación orientada a objetos (POO) se fundamenta en principios básicos, como la abstracción, la encapsulación, la herencia y el polimorfismo, que permiten construir sistemas organizados y reutilizables. El encapsulado de información es crucial, ya que restringe el acceso directo a los datos de un objeto, protegiendo su estado interno y facilitando la modularidad del código. En POO, es esencial distinguir entre una clase, que define propiedades y comportamientos comunes, y un objeto, que es una instancia concreta de una clase. Asimismo, es importante diferenciar entre variables, que pueden referirse a valores o a objetos, y los objetos mismos, que tienen estado y comportamiento definidos.

Además, reconocer los tipos de relaciones entre clases, como la herencia y la composición, es vital para estructurar correctamente el código orientado a objetos. Documentar las clases adecuadamente con herramientas como Doxygen es fundamental para mantener la claridad y la accesibilidad del código, permitiendo a otros desarrolladores entender y utilizar las clases de manera efectiva. La documentación automatizada facilita la colaboración y mantenibilidad en proyectos de software.

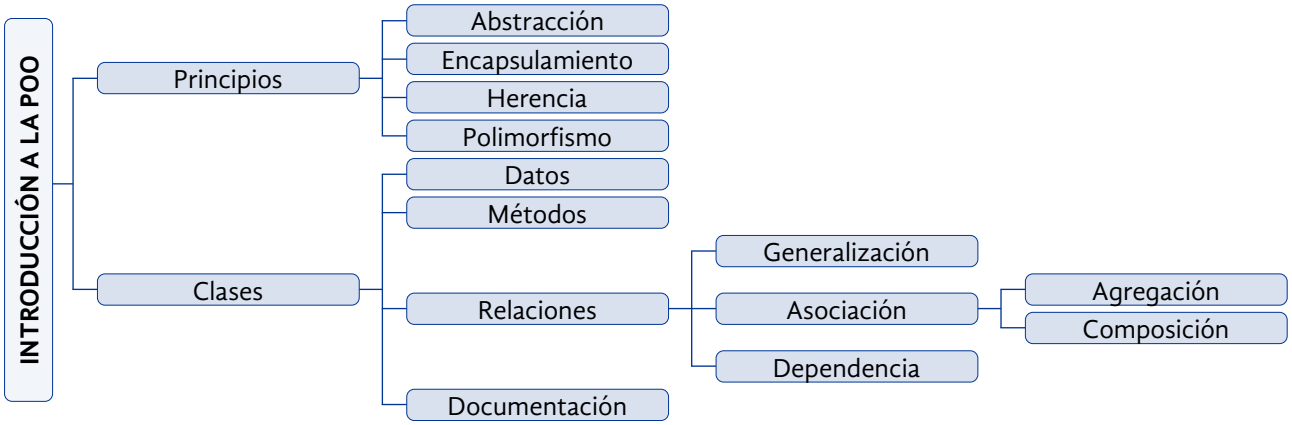

Fig.20. Esquema resumen del tema.

Resolución del caso práctico de la unidad

A continuación se muestra una posible resolución del caso práctico de la unidad:

```
public class Libro {
    String titulo;
    String autor;
    int añoPublicacion;

    public void mostrarInformacion() {
        System.out.println("Título: " + titulo);
        System.out.println("Autor: " + autor);
        System.out.println("Año de Publicación: " + añoPublicacion);
    }
}
```

Fig.21. Clase Libro.

/ 10. Bibliografía

Bhargava, A. Y. (2019). *Algoritmos: Una guía ilustrada para programadores y curiosos.* Madrid: Ediciones Anaya Multimedia (Grupo Anaya).

Caro, C. M.; Ramos, A. N., & Barceló, A. V. (2002). *Introducción a la programación con orientación a objetos*. PRENTICE HALL.

Haro, C. (2021). *Algoritmia: Razonar para crear*. Ediciones Eni.

Gómez, A., Fernández, E., Carrión, J., & Blanco, V. (n. d.). *Relaciones entre clases*. Recuperado de: http://elvex.ugr.es/decsai/java/pdf/3c-relaciones.pdf

Manuel, P. G. J. (2022). *Entornos de desarrollo*. Ediciones Paraninfo, S. A.

Ojeda, F. C. (2001). *Introducción a la programación*. Anaya Multimedia.

Schildt, H. (2018). *Java: The complete reference* (11th ed.). McGraw-Hill Education.

Vicente, E. (2019). *Introducción a la programación informática: Una excelente guía visual que explica, Paso a paso, Desde el Código Binario Hasta la Creación de Juegos.* Coordinación editorial de la Edición Española Elsa Vicente. Dorling Kindersley.